National University of Science and Technology
POLITEHNICA Bucharest
Faculty of Automatic Control and
Computers Computer Science and
Engineering Department

Diploma Project

POETGPT - A GENERATIVE MODEL FOR ROMANIAN AUTHOR-STYLIZED POETRY

Dobre Emilia Iliana

Thesis advisors:
Prof. dr. ing. Mihai Dascălu

BUCHAREST

2025

Contents

Sinopsis 3

Abstract 3

1. Introduction 4

1. 1. Context 4

1.2. Problem Statement 4

1.3. Objectives 5

1.4. Thesis Structure 5

2. State-of-the-art 6

2.1. Motivation and Problem 6

2.2. Literature: Generative and Classification Models for Text 7

2.3. Romanian LLMs and Resources 9

2.4. Poem Evaluation Techniques 10

3. **Method** 11

3.1. Corpus 11

3.1.1. Collection and Preprocessing 11

3.1.2. Dataset for Generation 12

3.1.3. Dataset for Classification 12

3.1.4. Data conversion for training the models 15

3.1.5. Descriptives 15

3.2. Neural Architecture 18

3.2.1 Tokenization 18

3.2.2. Generative Model: RoGPT2 19

3.2.3. Classification Model: XLM-RoBERTa 23

3.3. Performance Metrics 26

3.3.1. Metrics for Poetic Style Classification 26

3.3.2. Metrics for Poetry Generation 27

4. Results 29

4.1. Results for fine-tuned XLM-RoBERTa 29

4.1.1. Training Monitoring 29

4.1.2. Inference and Evaluation Metrics 30

4.2. Results for fine-tuned RoGPT2 32

4.2.1. Training Monitoring 32

4.2.2. Inference and Evaluation Metrics 33

4.2.3. Style Accuracy 35

4.3. Qualitative Evaluation with human participants 38

4.3.1. AI-Generated vs. Real Poetry 38

4.3.2. Style Attribution 40

4.3.3. Conclusion of the study 41

5. Discussion 42

5.1. Performance Comparison 42

5.1.1. Generative models 42

5.1.2. Aablation Study: Poetry Style Classifier 43

5.2. Limitations 48

5.2.1. Dataset limitations 48

5.2.2. Architectural limitations 48

5.2.3. Stylistic and conceptual limitations 49

6. Conclusions and Future Work 50

6.1. Future Work 50

6.2. Conclusions 51

References 52

Sinopsis

Acest proiect explorează posibilitatea generării automate de poezie în limba română prin intermediul unei arhitecturi duale — PoetGPT — care combină un model generativ (RoGPT2) cu un clasificator stilistic (XLM-RoBERTa). Scopul este generarea de poezii în stilul a șase autori canonici români și validarea automată a stilului literar. Lucrarea propune și o schemă de evaluare inovatoare, care folosește clasificarea stilistică.

Sistemul este evaluat atât calitativ, cât și cantitativ, printr-un studiu de caz asupra prompturilor și un ablation study detaliat al clasificatorului. Rezultatele arată că acuratețea stilistică poate fi crescută semnificativ prin optimizarea arhitecturii și integrarea vectorilor stilistici.

Lucrarea evidențiază limitările actuale ale modelelor NLP în ceea ce privește creativitatea și propune direcții de viitor care includ învățarea criticii literare și integrarea metadatelor culturale în generare. PoetGPT oferă nu doar un model funcțional, ci și o reflecție asupra modului în care inteligența artificială poate simula sau înțelege actul poetic.

Abstract

This thesis addresses the generation of Romanian poetry and proposes a dual-architecture for the task PoetGPT, which combines a fine-tuned generative model (RoGPT2) with a stylistic classifier (XLM-RoBERTa). The goal is to generate poems in the style of six Romanian authors and to validate their stylistic fidelity automatically. The project introduces an innovative evaluation pipeline based on style classification.

The system is evaluated both qualitatively and quantitatively, through prompt design experiments and an extensive ablation study on classifier variants. Results show that stylistic accuracy can be significantly improved by integrating handcrafted style vectors.

The thesis also discusses the limitations of current NLP models in simulating creative expression and proposes future directions that involve training on literary criticism and embedding cultural metadata. PoetGPT not only offers a working pipeline but also contributes to the theoretical conversation on whether creativity can be mathematically modeled, and to what extent machines can emulate poetic voice.

1 Introduction

As artificial intelligence has improved at an accelerating pace, language models have reached human-level skill at a dazzling number of language processing benchmarks. It was an eventuality that these systems would be asked to perform creative works—the truest proxy for machine intelligence. Poetry generation becomes a mirror of AI's cognitive limits: Can an algorithm truly express emotion, artistic intent, or stylistic subtlety? Today's models can generate hyper-realistic human faces, videos, and fluent conversations, but when it comes to art, the goal is no longer coherence — it is expression. This thesis begins with a fundamental question: Can AI genuinely create poetry, or is it merely performing imitation at scale? And if it cannot, what exactly is missing?

1.1 Context

Natural Language Processing (NLP) has made significant progress in recent years in applications like text classification, machine translation, question answering, summarization, and named entity recognition. However, when it comes to creative language tasks, like poetry writing, progress has been slower due to the dense, metaphorical, rhythmically structured, and deeply stylistic nature of poetry. Recent advances in large language models (LLMs), such as GPT and BERT, have opened up new pathways for creative text generation. This thesis examines their capabilities and limitations in the context of Romanian poetic language, intending to understand whether stylistic features can be learned and controlled, and whether poetic voice can be recognized computationally.

1.2 Problem Statement

Although recent advances in generative language modeling have enabled poetic generation in English and other high-resource languages, Romanian remains underrepresented in this space. There is no existing system capable of generating Romanian poetry in the style of multiple authors while also validating stylistic fidelity. Most current models are limited to surface-level fluency and lack mechanisms for evaluating literary voice. This project addresses that gap by proposing PoetGPT. This system combines generation and classification to simulate and validate Romanian poetic style, exploring the capabilities of language models in the context of Romanian poetic language.

1.3 Objectives

The goal of this thesis is to test whether an LM can produce poetry in the distinctive style of different Romanian poets. The project developed a two-tap (generation and evaluation) model that enables the generation of both grammatical and fluent text, an approximation of poetic voice and style, and an estimate of the stylistic identity of poets. Apart from building this system, the thesis also seeks to further examine its performance, limitations, and implications for the general issue of creativity in AI. Second, the project aims to enrich Romanian NLP resources by experimenting on fine-tuning methods, corpus aggregation techniques, and rating-styler practices.

1.4 Thesis Structure

This thesis is structured into six chapters that provide a comprehensive study of AI-generated Romanian poetry. Chapter 1 describes the universal background, the research problem, and the corresponding domain of creative language modeling. Chapter two reads as a state-of-the-art research, structured from a comprehensive approach point of view and focused on the reasons why the Romanian poetic domain has been selected as the topic of interest. It then discusses related work on poetry generation and stylistic classification before synthesizing results from the generative and classification models mentioned. Additionally, this chapter provides information about Romanian language resources and the current work being developed in the field of Romanian poetry. The final section of Chapter 2 outlines the contributions made by this work, including a new evaluation technique, along with further justification for utilizing these technologies. In chapter 3, the following sections introduce the methodology, including dataset construction, preprocessing, model structure, training configuration, and performance metrics. Section 4 details the results, visual analysis, and interpretation (for both the generator and classifier), and describes the inference pipeline. It also includes a human evaluation experiment in which people judge poems based on their authorship, given a set of generated poems. The performance analysis is performed in Chapter 5, comparing three different architectures, performing an ablation study of the classifier, and taking a deeper look at the system limitations. Chapter 6: Future work suggestions and closing remarks presents directions for continued research and concludes the thesis by reflecting on the broader significance of poetic generation.

2 State-of-the-art

In this section, the current state of the field will be examined, along with existing poetry generation systems, following a discussion of the tools for stylistic modeling and the datasets commonly used in this domain. This analysis will highlight the need for a new approach in the context of language generation models, providing concrete examples of what is currently missing from the market and research.

Several poetry generation tools and projects have been developed in recent years. Deep-speare (Lau et al., 2018), for instance, focuses on generating English sonnets in the style of Shakespeare, capturing rhyme and meter, but lacks stylistic diversity or support for low-resource languages. Verse by Verse (Uthus et al., 2021), a Google project, offers user-guided poetic completions inspired by American poets but does not attempt whole poetic generation or style validation. In Romania, initiatives like Sofetscu's project—LLaMA-Eminescu—represent early attempts to model Romanian poetic style, trained solely on Mihai Eminescu's work. However, these are single-author systems without stylistic evaluation capabilities or broader language modeling goals. Projects like Wang et al. (2022) demonstrate the feasibility of author-style classification using statistical models but focus only on Chinese classical poetry and do not involve generation. Overall, current solutions lack support for multi-author style modeling, automatic stylistic validation, and adaptability to under-resourced languages, such as Romanian. Complementary to these efforts, Ovidiu Calin (2020) explores, in his article for Eastern Michigan University, a statistical and neural approach to poetry analysis and generation, focusing on both English and Romanian and offering early insights into cross-language stylistic modeling and evaluation.

2.1 Motivation and Problem Framing

There is a noticeable gap in the study of how generative models behave and analyze the stylistic particularities of Romanian poetic language. This lack of research, coupled with the absence of large, structured, publicly available datasets of Romanian poetry labeled by author, and the fact that machine learning models have not been explicitly trained to understand or reproduce Romanian poetic style, presents a clear need for the proposed project.

Most poetry generation systems developed so far have primarily focused on English and rely on either stylistically neutral texts or models fine-tuned on the work of a single author. This limitation makes them unsuitable for multilingual tasks and ineffective for capturing the nuanced stylistic differences between authors. The challenge lies in understanding what makes a writer's voice truly distinctive, what gives their poetry the tone and rhythm that we, as humans, intuitively recognize as authentic and familiar.

Despite recent advancements in large language models, the task of poetry generation in any language remains highly challenging because the models are primarily trained on prose. Prose and poetry differ significantly, and current systems often fail to capture the rhythmic, symbolic, and stylistic structures that are essential to poetry. Existing models tend to focus on isolated aspects, such as a single literary movement, a few stylistic particularities, or the work of a single author. This thesis makes a novel contribution: a system capable of both learning and validating poetic style, introducing a new form of evaluation. Traditional evaluation metrics such as BLEU and ROUGE are poorly suited for assessing poetic quality and style, as they emphasize surface-level similarity rather than true stylistic fidelity to the target poet.

Another important constraint is the lack of high-quality, author-labeled poetic data, particularly for the Romanian language, which significantly restricts both the training and evaluation phases. To fill these gaps, the project focuses not only on the automatic production of poetry in Romanian but also on its stylistic evaluation using an approach that goes beyond traditional generation evaluation tools.

2.2 Literature: Generative and Classification Models for Text

Deep-Speare

Deep-Speare is a neural model designed to generate English sonnets in the style of Shakespeare, integrating three key components: a language model that captures poetic content, a meter model that enforces iambic pentameter, and a rhyme model that ensures appropriate rhyme patterns. It is trained on a curated dataset of approximately 2,600 sonnets sourced from Project Gutenberg. The language model is a sequence-to-sequence LSTM with attention. In contrast, the meter model operates at the character level to learn syllabic stress patterns without relying on external phonetic dictionaries. The rhyme model operates similarly on character sequences, calculating rhyme likelihood via cosine similarity, which enables rhyme prediction in a language-agnostic manner. Although Deep-Speare achieves high accuracy in generating metrically and rhythmically consistent verse, as confirmed by both crowdworker and expert evaluations, it falls short in generating emotionally compelling or semantically rich poetry. Furthermore, the model is limited to English and cannot be directly applied to Romanian. It cannot also distinguish between distinct poetic voices, as the main focus of the project is on understanding structural features such as rhyme and meter. This makes Deep-Speare less suitable for tasks involving stylistic diversity and author-specific generation. (Lau et al., 2018)

Wang's Research— Chinese Poetry Style Classifier

Wang (2022) also offers a valuable method and source of inspiration for PoetGPT. He employs classic text classification algorithms to group poems by style. The study aims to automatically identify the author of a given poem by analyzing stylistic features extracted from the text. The authors employ a range of machine learning models, including Random Forests, Naive Bayes, and Support Vector Machines, which are applied to a dataset of classical Chinese poems labeled by author. The methodology follows a standard natural language processing pipeline, including text preprocessing, feature extraction (such as lexical frequency, sentence length, and symbolic use), and classification. Among the tested algorithms, Random Forest achieved the highest performance, with classification accuracy exceeding 90%. This work demonstrates that poetic style can be captured and discriminated using relatively simple models, without requiring deep neural networks. The study is only suitable for Chinese classical poetry and may not generalize to diverse and modern poetry. However, within these limitations, the outcomes demonstrate that the author's voice is indeed statistically modelable and detectable, opening up perspectives for stylistic analysis across different languages, such as Romanian. (Wang, 2022)

Verse by Verse

Another notable example is Verse by Verse, a Google project that supports interactive poetry composition. Instead of generating full poems autonomously, the system assists the user line by line by suggesting verses in the style of selected American poets. It uses a two-step architecture: a Transformer-based generative model fine-tuned for each poet creates a large set of possible lines offline, while a dual-encoder retrieval model selects the best suggestions in real time based on the previous line written by the user. While the results are not indistinguishable from human poetry, the system achieved promising scores in human evaluations for readability and aesthetic appeal. However, it does not attempt full poem generation, nor does it include any automatic style validation. Compared to this, the current thesis aims to go further by generating complete poems in the style of multiple Romanian authors and validating their style through automatic classification. (Uthus et al., 2021)

Ovidiu Calin's research on poem generation

Finally, one more recent work related to the project is described in Ovidiu Calin's research (2020), which employs and combines classical statistics and machine learning for the analysis and generation of poetry. Simple methods such as letter frequency, entropy, and N-gram models are employed to explore how poetic texts diverge from regular ones. These tools allow him to quantify how "organized" or "predictable" a poem is, measures that often jibe with a poet's style.

In the second half, he teaches a recurrent neural network (RNN) how to create new poetry in the style of Lord Byron.

To evaluate how close the generated poems are to the original ones, he calculates the difference in letter distribution using a method called Kullback–Leibler divergence. Even though his model is simpler than modern transformer-based systems, it shows how combining statistics with deep learning can help us both generate and evaluate poetic style, especially in under-resourced languages like Romanian. (Calin, 2020)

2.3 Romanian LLMs and Resources

While large language models (LLMs) for Romanian are still emerging, several notable efforts have been made in recent years. Models such as RoGPT2, RoBERT, RoLLama, and RoGeMa have been trained or adapted for various Romanian natural language processing (NLP) tasks. RoGPT2 is a Romanian adaptation of GPT-2, suitable for text generation but not fine-tuned for poetic style. RoBERT and its improved variant, XML-RoBERTa, are transformer-based encoders widely used in classification and token-level tasks such as NER or sentiment analysis. RoLLama and RoGeMa represent newer initiatives built on Meta's LLaMA architecture, aimed at expanding generative capacity in Romanian.

Limitations

Despite their potential, these models share essential limitations:

- None are trained explicitly in poetic language.
- There is no fine-tuning for stylistic generation or literary tone.
- There is a clear lack of high-quality, author-labeled Romanian poetry datasets.

These limitations restrict the performance of existing Romanian language models (LLMs) in creative writing tasks. While some models (e.g., RoGPT2) are technically capable of generating fluent text, they lack stylistic sensitivity and perform poorly on domain-specific outputs such as poetry. This is particularly problematic for Romanian, where stylistic nuances, diacritics, and poetic rhythm play a key role in authorial voice.

Eminescu LLaMA and the project's limitations

A compelling recent development in Romanian poetry generation is presented by Softescu's project "Fine-tuning LLaMA to Recreate Eminescu's Literary Style" (2025). This effort adapts a base LLaMA model via LoRA fine-tuning on the public domain works of Mihai Eminescu, leveraging Google Colab with Python. The training data consist of Eminescu's poems and prose, with special attention given to encoding Romanian diacritics and 19th-century literary language. The resultant model effectively reproduces stylistic elements characteristic of Eminescu, including rich natural imagery, romantic and contemplative tones, and thematic consistency, with the larger 70 B-parameter version showing marked improvements in

coherence. However, it lacks multi-author stylistic diversity and does not include any automatic style-validation mechanism.

Additionally, there are no official evaluations, such as human judgments or quantitative measurements, and the model's effectiveness depends on the quality and quantity of the Eminescu corpus. While this work is an encouraging step for Romanian poetic AI, it highlights lingering challenges in stylistic validation, dataset limitations, and the absence of multi-author generative features.

2.4 Poem Evaluation Techniques

Evaluating the quality of generated poetry is a complex task because poetic language often extends beyond semantic clarity and incorporates subjective dimensions, such as rhythm, emotion, metaphor, and stylistic nuance. In computational terms, two major approaches are commonly used to assess generated text: automatic metrics (based on textual similarity or probability) and classifier-based evaluation (based on stylistic or semantic alignment).

Limitations

The traditional automatic metrics for language models, such as BLEU, ROUGE, and perplexity, are useful for evaluating syntactic fluency and token-level coherence. However, they face limitations when attempting to capture deeper features, including style imitation.

To address these limitations, this thesis introduces a second layer of evaluation: style classification. The idea is to use a supervised machine learning model trained to recognize the poetic style of individual Romanian authors. The model used is XLM-RoBERTaFroSequenceClassification, a transformer-based multilingual encoder pretrained on massive datasets, including Romanian, and fine-tuned here to classify generated stanzas into one of the six target authors.

Innovation

This approach supports a more nuanced validation of style by checking whether the classifier recognizes the generated poem as matching the intended author’s voice. When the model achieves high classification accuracy, it suggests a strong alignment with the stylistic features of that author. This method reflects recent developments in NLP, where transformer-based models such as BERT are increasingly used to evaluate aspects like genre, authorship, and tone in generated texts.By integrating both conventional evaluation metrics, such as BLEU, ROUGE, and perplexity, with a stylistic classification layer, the evaluation strategy proposed in this thesis provides a more complete and context-aware assessment of poetic text generation.

3 METHOD

This chapter outlines the methods used to develop and evaluate the proposed system for generating Romanian poetry and validating its stylistic aspects. It consists of details for the preprocessing of the corpus, the model architecture for both generation and classification, and metrics used to evaluate performance. The focus is on presenting the system's components in a clear and structured manner, along with the key design choices that support the overall approach.

3.1 Corpus

To train and evaluate models for poetry generation and stylistic classification, a dedicated corpus of Romanian poetry was compiled and processed. This section describes the data sources, the preprocessing steps applied, and the statistical characteristics of the resulting dataset.

3.1.1 Collection and Preprocessing

The first stage of this project involved gathering and preparing a Romanian poetry dataset that would serve as the foundation for both text generation and style classification tasks. In the absence of any publicly available, large-scale poetic corpus labeled by author, the data was collected manually and semi-automatically from various open-access sources. These included online cultural archives, literary journals, educational repositories, and scanned PDF volumes that were digitized using OCR[1] or extracted manually.

Formatting Problems

Once collected, the texts displayed numerous inconsistencies, particularly about diacritics and formatting. Many of these issues were due to outdated character encodings such as Windows-1250 or ISO-8859-2, as well as errors introduced during OCR processing or web scraping. Among the most common problems were historical forms of Romanian diacritics—such as ‘ș’ and ‘ț’ (with cedilla)—which differ from the correct Unicode standard. characters ‘ș’ and ‘ț.’ Although visually similar, these are interpreted differently by NLP tokenizers and can result in incorrect vector representations during training.

Solution

To address this, a custom Python script was used to detect and replace non-standard characters with their correct Unicode forms.

[1] https://ploomber.io/blog/pdf-ocr/

Additional normalization steps targeted typographic artifacts often left behind by text editors or PDF converters, such as wise quotes (“ ”), long dashes (—), apostrophes (’), tildes (~), and other formatting symbols. These were either standardized or removed, depending on context. In neural language processing, even minor inconsistencies in encoding or punctuation can disrupt tokenization, lead to vocabulary mismatches, or reduce model performance. By providing clean and orthographically correct input, the models were given reliable data from which to learn stylistic patterns effectively.

3.1.2 Dataset for Generation

To train the language model, a dedicated dataset was constructed using works from six prominent literary figures: Mihai Eminescu, Lucian Blaga, George Bacovia, Tudor Arghezi, Ana Blandiana, and Marin Sorescu. These authors were selected for their stylistic distinctiveness, historical significance, and representativeness across various literary periods and poetic sensibilities. The dataset was saved in CSV format, with each row corresponding to a single poem, including metadata such as the author's name, the poem's title, and the full text of the poem.

Preprocessed dataset for training

The data was preprocessed as described to ensure consistency and suitability for input into a generative language model like RoGPT2. To prepare the data for fine-tuning RoGPT2, the poems were serialized into a plain-text format using a custom Python script. The script structured each poem as follows:

```
'<|autor|>', '<|titlu|>', '<|poezie|>', '<|endofpoem|>'
```

This format explicitly marks metadata and poem boundaries, enabling the model to learn structured prompts and conditioning. The delimiter tags were inspired by approaches used in instruction-tuned language models, facilitating better control over stylistic prompts during inference.

3.1.3 Dataset for Classification

Stanza instead of a whole poem

While the dataset for generation treated each poem as a single unit, the classification task required a finer granularity of analysis. To this end, each poem was split into stanzas, which were treated as the basic units of classification. A stanza was defined as a group of verses separated by blank lines or ending in terminal punctuation (period, question mark, exclamation mark, ellipsis). This approach reflects poetic structure and provides a manageable unit length for model training.

Normalization and dataset balancing

Stanzas with fewer than four lines were discarded due to insufficient stylistic content. Those between four and twelve lines were retained without modification. Longer stanzas were split using a custom segmentation algorithm that aimed to create smaller, balanced units within the 4- to 12-line range. For poems containing unusually long verses or segments that could not be naturally divided, typically because they lacked precise terminal punctuation (periods, exclamation marks, question marks, or ellipses), a punctuation mark was artificially appended at the end of the segment. This step ensured consistency in stanza boundaries and prevented tokenization errors during training, primarily when the absence of punctuation would have caused unintended merging of stylistically distinct units.

After normalization, a new dataset was compiled, which preserved stanza order, authorship, and poem titles. This version was then filtered and balanced to mitigate class imbalance. Specifically, only a quarter of the stanzas were selected for Mihai Eminescu, while all available stanzas for the other five authors were retained.

Handcrafted Stylistic Feature Vectors

Besides the stanza, after trial and error, the best results for the classifier were achieved by adding extra information: a list of 7 stylistic features. The stylistic vector was the solution and consists of information about the:

- Number of verses in the stanza.
- Average verse length, measured in characters.
- Ratio of adjectives, verbs, and nouns relative to the total number of tokens.
- Number of punctuation tokens, as a proxy for rhythm or expressiveness.
- Rhyme pattern type is a categorical feature indicating rhyme structure.

POS Tagging and spaCy

These features were extracted using the *spaCy*[2] natural language processing library, optimized for efficiency, precision, and speed, in grammatical analysis. For the Romanian language, it offers the pre-trained linguistic model `ro_core_news_sm`, which can tokenize the text into phrases and words, grammatically analyze the text, identify the function of each word (POS—Part-of-speech tagging), and generate syntactic relations between words. The model decides for each token if it is a noun, an adjective, an adverb, a pronoun, a preposition, a conjunction, or a punctuation mark, which is essential for stylistic particularities.

[2] https://spacy.io/usage/linguistic-features

Detecting Rhyme Structure

One final feature aimed to capture poetic structure through rhyme. A custom rule-based function was implemented to analyze the last syllable of each line and match them into common rhyme patterns. The syllable works with a regex-based vowel match, not a phonetically correct approach. The logic compared pairs of line endings using string matching, as shown below:

```
1. def rhymes(a, b): return endings[a] == endings[b]
```

Listing 1 — Rhyme

If four lines are present, the function assigns one of the rhyme types listed below:

Code	Rhyme Type	Pattern	Example
0	Paired rhyme	AABB	L1-L2, L3-L4
1	Cross rhyme	ABAB	L1-L3, L2-L4
2	Embraced rhyme	ABBA	L1-L4, L2-L3
3	Monorhyme	AAAA	All lines are the same
4	No rhyme	-	No match at all
5	Mixed rhyme	Irregular like AABC	Some match
6	Other (fallback)	-	< 4 verses stanza

Table 2 — Type of Rhyme

This value was treated as a categorical feature and added to the stylistic vector.

```
1. Stanza:
2. De aceea eu mă emoționez,
3. și timp de câteva ore
4. le povestesc
5. ce frumos a murit sufletul meu peste zi.
6. Vector:
7. [6, 16.0, 0.037, 0.0370, 0.1481, 2, 4]
```

Example 3 — Stanza and style

Dataset Splitting: Training, Validation, and Test Sets

For model training, the labeled data was split into training (70%), validation (15%), and test (15%) subsets using stratified sampling.

3.1.4 Data conversion for training the models

Before training the neural models, the structured poetic data (stored in .csv format) was first loaded into a **`Pandas DataFrame`**[3], a flexible tabular format suitable for text processing, cleaning, and feature extraction. The `DataFrame` allowed the integration of metadata (author, title), raw text (poem or stanza), and additional stylistic features engineered specifically for the classification task.

Once all preprocessing was complete — including label encoding, stanza segmentation, stylistic vector construction, and normalization for the classification model, as well as token-specific addition for the generation model — the data was converted into a **`PyTorch Dataset`**. This way, the input from tabular data was transformed into a format compatible with PyTorch, allowing its use in transformer-based models such as RoGPT2 and XLM-RoBERTa. It provided the model with informative and coherent, well-structured data throughout training.

3.1.5 Descriptives

Generation dataset

For the generation dataset, a total of 918 poems were collected, with approximately 150 to 153 poems per author. Every poem has a unique length and complexity of structure, reflecting the distinct stylistic patterns of its poet. These variations are necessary for the model to capture the average structure and length of an author's poetic voice. The key features of this dataset are summarized and presented in the following tables and graphs.

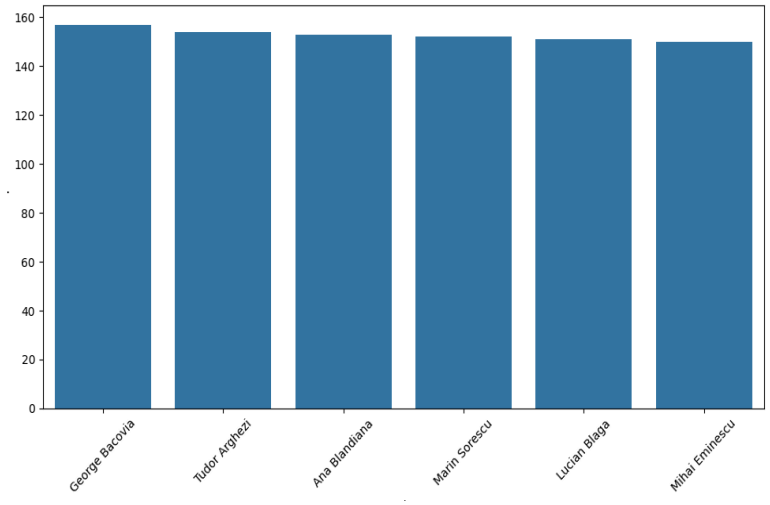

Figure 4 — Poems per author distribution

Figure 4 shows that the poetry generation dataset was well-proportioned for the six chosen authors, providing nearly 150 poems from each of them. This compensation is crucial for avoiding training bias and learning stylistic features uniformly.

[3]https://pandas.pydata.org/docs/user_guide/10min.html

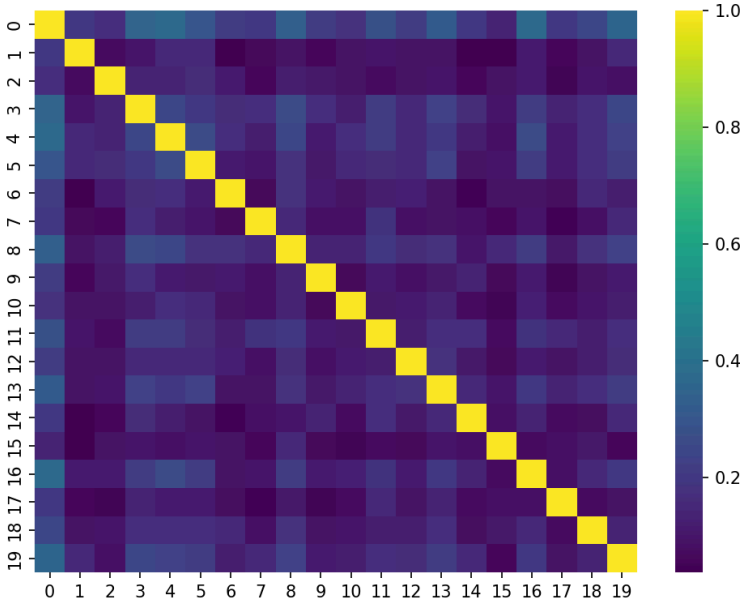

Figure 5 — Stylistic similarity

The heatmap in Figure 5 displays similarity between poems, calculated using TF-IDF vectors. The diagonal line is bright yellow, indicating perfect self-similarity—each poem is identical to itself. This is expected in cosine similarity matrices. The rest of the heatmap shows lower similarity scores between different poems, marked as darker shades. This pattern confirms that the dataset is lexically diverse, making it suitable for training a stylistically rich and complex generative model.

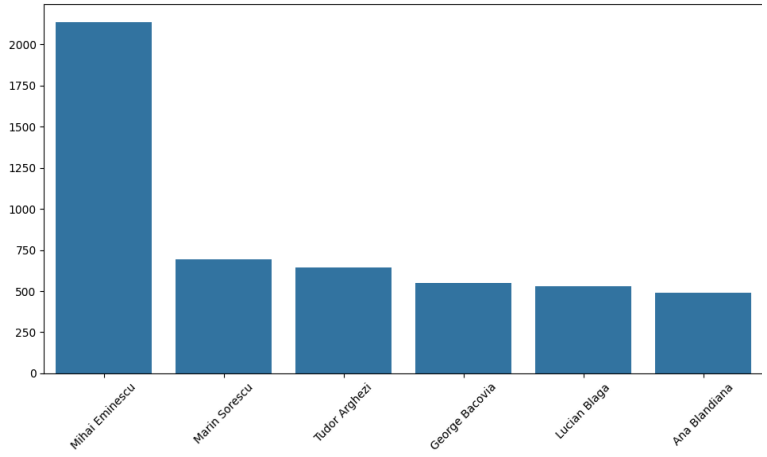

Figure 6 — Poem length distribution

The graph in Figure 6 shows the length of each poem for each author. Mihai Eminescu, as expected, contributed with long texts, while the others provided much more compact and controlled styles. These outlying texts did not require any special preprocessing.

They were the same in terms of author style and text quality. Eminescu doesn't condition the generator, though he does have more text; the generator simply tends to generate longer poems in the case of Mihai Eminescu, naturally.

Classification dataset

For the classification dataset, as detailed in the section above, each poem was split into stanzas and normalized to 4–12 lines, resulting in a total of 2,667 stanzas —a balanced dataset across authors to ensure fair learning conditions. The two charts below show the ultimate selection of stanzas and their formal aspects.

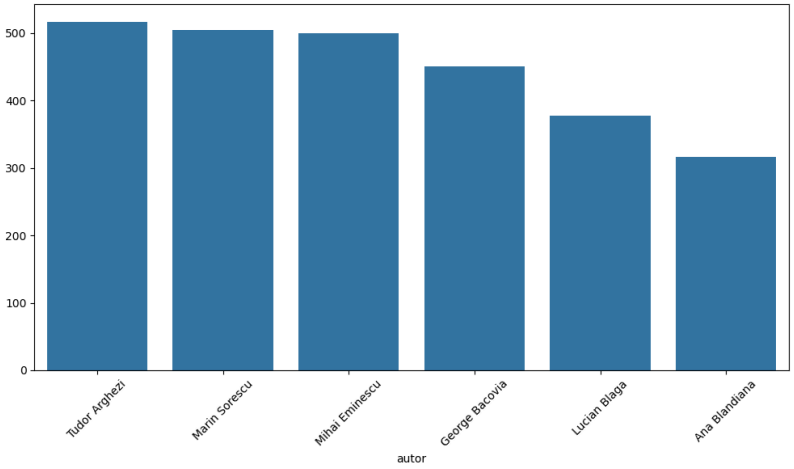

Figure 7 — Stanza per author distribution

Figure 7 illustrates that both classes contain a similar number of stanzas; the ratio of the largest to the smallest class is less than 1.6 times. This ensures that the classifier does not overfit to the overrepresented authors, thereby making learning fair for all style categories.

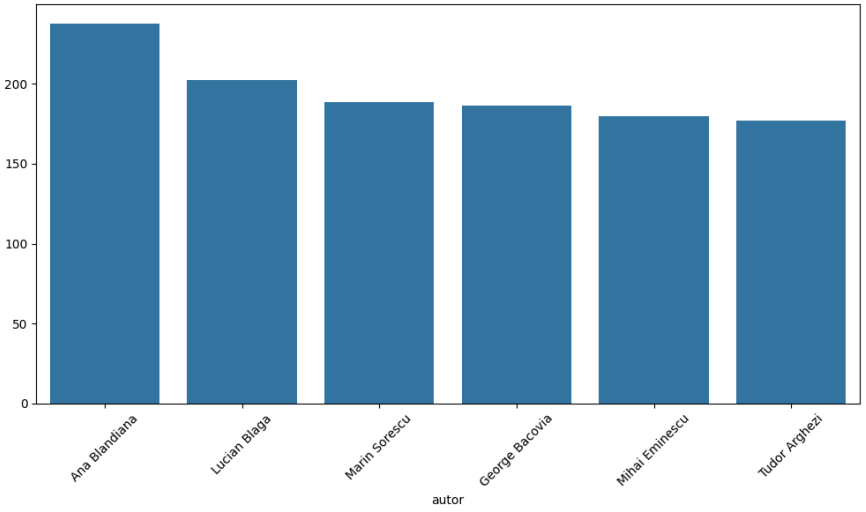

Figure 8 — Average stanza length per author

The chart in Figure 8 displays the median number of characters per stanza for each author, and it is readily apparent that Blandiana has the longest stanzas. Authors with longer stanzas tend to write more elaborate, descriptive, and philosophical verse, whereas those with shorter stanzas have a much more rhythmic and concise style.

3.2 Neural Architecture

This section presents the complete neural architecture of PoetGPT, the system integrates two deep learning components: a generative language model responsible for producing poetic text, and a classification model that ensures the generated content aligns with the stylistic signature of a specific Romanian poet. To ensure stylistic consistency, the system follows this workflow: it receives a prompt (including the poet's name and poem title), generates a poem, classifies it stylistically, and then either accepts or regenerates the output based on the classification confidence.

The subsections below detail the tokenization process, loss functions used in training, the architecture of the generation and classification models, and a deeper structural breakdown of each neural network component.

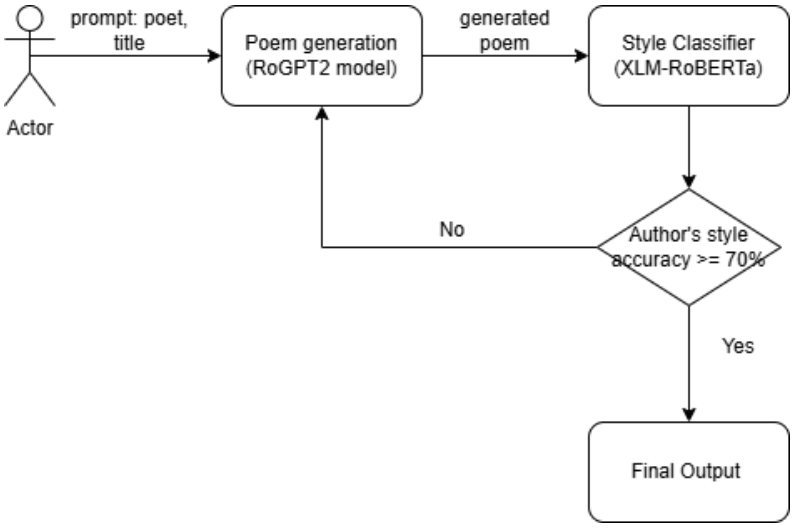

Figure 9 — System Workflow Diagram

3.2.1 Tokenization

The first step in any neural language modeling task is tokenization. This process plays a crucial role in both proposed systems. It involves converting text into a numerical representation that the models can understand.

For both models, tokenizers based on Byte-Pair Encoding[4] were chosen. BPE splits words into subword units, allowing the model to handle rare or unseen words by decomposing them into known components. These components become more frequent, and the model starts learning the language's structure more effectively, thereby avoiding being stuck on rare words, a salient aspect when modeling artistic language.

[4] https://huggingface.co/learn/llm-course/chapter6/5

It is more efficient than character or word tokenization, and it reduces the dimension of the vocabulary, offering a perfect foundation for the model to learn. Each token receives a given ID, its numeric equivalent based on a predefined vocabulary of the model, which is found in the *vocab.json* file.

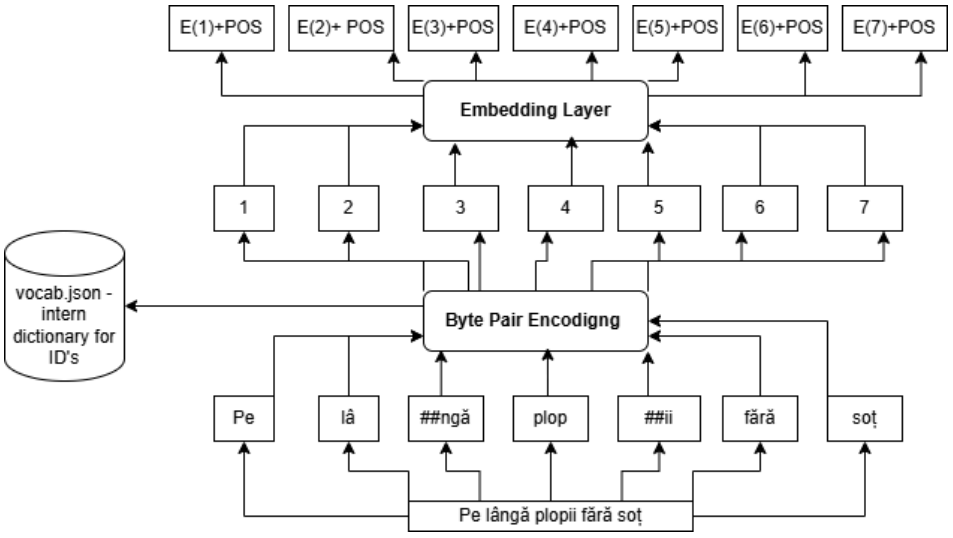

Figure 10 — BPE workflow

3.2.2 Generative Model: RoGPT2

The generative component of the system is based on the RoGPT2-medium model, a Romanian language adaptation of the original GPT-2 architecture. It follows a decoder-only transformer structure and is trained in an autoregressive manner, predicting the next token given the previous ones, meaning it generates text but does not comprehend it. For predicting the next token, it uses masked self-attention, meaning that each token only sees the previous tokens.

Reasons for fine-tuning RoGPT2

This architecture was selected for its ability to generate fluent text that is also contextually appropriate in Romanian, which makes it highly suitable for tasks involving creative writing, such as poetry generation. RoGPT2 builds on the GPT2[5] architecture, known for its autoregressive capabilities, and was trained on a 17GB Romanian corpus, including OSCAR, Wikipedia, news, books, and parliamentary debates. Among the three available configurations (base, medium, and large), the medium configuration was chosen for this work, as it strikes a reasonable trade-off between quality and training time.

[5] https://huggingface.co/openai-community/gpt2

Another reason to select RoGPT2 is the training with Adam optimizer. This is particularly important in creative writing tasks, as long sequences must be addressed, and subtle stylistic patterns must be captured for flexible optimization. Adam achieves this by modifying the learning rate for every parameter in the model by different factors to achieve a stable and realistic learning process.

RoGPT2 Architecture

RoGPT2 is built on a decoder-only Transformer architecture, designed explicitly for autoregressive text generation. Unlike encoder-based models that process entire input sequences simultaneously, RoGPT2 predicts text token by token, using only the past context to generate the next word. This is particularly useful for creative writing tasks, such as poetry, where each new word or verse depends strongly on the flow and rhythm of the previous ones. The autoregressive property is mathematically described as:

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

The model's architecture consists of the following layers:

1. **Token and Positional Embedding**
   In the first layer, each input token is first transformed into a dense vector (embedding), where each value is non-zero, unlike one-hot vectors. A positional embedding is added to encode the position of each token in the sequence. This is crucial for maintaining word order and poetic structure, since Transformers don't have built-in sequence awareness.

2. **Transformer Decoder Blocks (Stacked Layers)**
   RoGPT2 medium has 24 stacked, identical Transformer Decoder blocks, and each one of them has three main components:

   - **Masked Multi-Head Self-Attention**: This is what enables autoregressive generation, as each token is allowed to understand only the relations with the tokens preceding it, which is essential for causal logic in text; this is what masked stands for. Moreover, the multi-head part means that the tokens learn multiple different ties with other tokens, such as rhyme, rhythm, repetitions, metaphors, or structural grammar rules. The role is to identify which of the anterior tokens are more relevant to generate the next one.

   - **Feedforward Neural Network (FFN)**: A two-layer MLP that applies non-linear transformations to each token representation individually, with ReLU activation.

This is particularly helpful because it generates more complex, abstract words, which are salient for imitating creativity in text. The information becomes more abstract due to the independence of each token processed, as it follows learning attention; this component serves as a tool for learning stylistic patterns.

- **Layer Normalization + Residual Connections**: Stabilize training, prevent vanishing gradients, and enable the network to operate more deeply. This allows RoGPT2 to be trained effectively over 24 stacked layers, enabling it to capture complex poetic structures and maintain stylistic consistency across long sequences.

3. **Output Projection + Softmax**

After the input undergoes byte-pair tokenization and is converted into a token, it then passes through the embedding layer, where it is transformed into a dense vector with position information. This vector enters the 24-block layered network, where the model learns the relations between tokens. Finally, the model generates the hidden state. This hidden state is a vector that represents what the model predicts will come next. Through linear projection, the final layer becomes a bidirectional vector of the same dimension as the dictionary, assigning a score to each word in the dictionary. Then, with Softmax, the score is converted into a probability distribution as the final output. At inference, the token sampled using top-k and temperature is chosen and appended to the sequence.

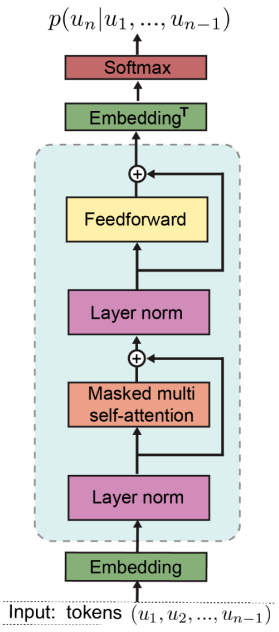

Figure 11[6] — GPT-2 architecture

[6] https://www.researchgate.net/figure/GPT-2-architecture-Heilbron-et-al-2019_fig1_358654229

Fine-tuning RoGPT2 for Poetry Generation

To allow the model to condition generation on author and title, a few custom control tokens were added: `<|autor|>`, `<|titlu|>`, `<|poezie|>`, and `<|endofpoem|>`. The embedding matrix and the tokenizer vocabulary were adjusted using the tokens. This simple modification instructs the model to generate text in a desired stylistic direction.

Fine-tuning was handled via HuggingFace's Trainer class. Internally, the *`.train()`* method performs multiple passes over the training data (epochs), using backpropagation to adjust the model's parameters.

```
TrainingArguments(
    output_dir="./poetgpt5",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    fp16=True,                          # mixed precision
    logging_steps=100,
    save_steps=500,
    save_total_limit=2)
```

Listing 12 — Training Generator

To support this adaptation process, each of the training parameters was carefully chosen and tested. The choice for batch size was set to 2 due to GPU memory constraints in the working environment, and mixed precision training (fp16=True) was enabled to reduce resource usage while maintaining performance.

The number of training epochs was empirically selected as five after testing with both fewer and more epochs. While three epochs led to underfitting, exceeding five epochs showed diminishing returns and signs of overfitting, especially given the moderate size of the dataset. Therefore, five was the "sweet spot" for the number of rounds, allowing the model to learn deeply enough without memorizing the data.

The model checkpoint was also persisted after a fixed period (every 500 steps) for the sake of traceability during training and for ease of recovery in case training needs to be interrupted. These parameters enabled fine-tuning to be stable and computationally efficient, all while working within a limited computational budget.

During this procedure, RoGPT2 gradually developed the course of stylistic, lexical, and structural features for particular Romanian poets. Therefore, conditioned on an input author and title pair, the model is capable of generating poetic material that is consistent and stylistically similar to the target author as measured by the stylistic discriminator in the next section.

3.2.3 Classification Model: XLM-RoBERTa

XLM-RoBERTa[7] is a bidirectional encoder Transformer trained to understand natural language in hundreds of languages, including Romanian, making it a multilingual model capable of comparing texts in different languages. The model is trained on an unsupervised learning method — Masked Language Modeling (MLM) — where the model learn to predict words missing from a text, understanding the context.

Reasons for fine-tuning XLM-RoBERTa

The model used in this work is XLMRobertaForSequenceClassification, a specialized version of the general XLM-Roberta base model, adapted for sequence classification tasks. Built on the XLM-RoBERTa encoder architecture, this variant adds a classification head on top of the final hidden state, allowing it to assign probability scores to multiple classes — in this case, the six Romanian poets. The model has been pre-trained with MLM on 2.5 TB of CommonCrawl multilingual data, which contains Romanian examples extracted from news, the web, and Wikipedia.

Additionally, although pre-training is not directly task-based, the inclusion of a wide range of natural Romanian content in the pre-trained models enables them to consistently capture the lexical and syntactic subtleties necessary for literary style classification. The model is a good choice because it is lightweight and can be fine-tuned on a relatively small dataset.

XLM-RoBERTa for Classification Architecture

The model performs best for tasks that require an understanding of context. It is designed to process the entire sequence of tokens and produce a contextualized representation for each of them, which is then used for classification problems.

Below are the main components of the neural architecture, explained layer by layer and why this matters in the context of validating the style of the poems:

1. **Tokenizer Layer and Embedding Layer**
   As explained in the sections above, the model utilizes BPE tokenization, which breaks words into subwords and maps them to an integer ID from the model's vocabulary. After that, it is transformed into a token embedding with a position embedding, the same process as the generative model.

2. **Transformer Enocdeer Blocks (Stacked layers)**
   XLM-RoBERTa models, including the sequence classification, all have 12 encoder block layers with the same three components:

[7] https://dataloop.ai/library/model/facebook_xlm-roberta-xxl/

- **Bidirectional Multi-Head Self-Attention**: Each token can interact with tokens before or after its position in the text, allowing the model to comprehend the global context of the stanza. This enables the model to gain a deeper understanding of the relationships from the beginning of the poem to its conclusion. All this data about the whole stanza is collected in the [CLS] vector that contains the meaning of the stanza.

- **Feedforward Neural Network (FFN)**: The tokens and the [CLS] are transformed, one by one via a neural network called position-wise FFN, built upon two fully connected layers, with a GeLU activation — a better choice for understanding tasks because it does not discard negative values, it attenuates them using a probabilistic curve allowing the model to preserve subtle lexical cues. This helps the model transform the token embeddings and context vector (CLS) into more abstract and descriptive versions. The model decides which word has a stylistic function: an adjective or a metaphor. Through repetition in these 12 layers, FFN helps refine these stylistic features, which are essential for learning a poem's style.

- **Layer Normalization + Residual Connections in the Encoder:** Similar to the generative model, this approach ensures stable training by preventing internal covariate shift. In contrast, residual connections help preserve and propagate essential features throughout the network.

  In the context of poetic classification, these mechanisms allow the model to retain subtle stylistic cues — such as tone, metaphor, or rhythm — across multiple layers, without losing their influence.

3. **Classification Layer and Output Layer**

   After passing through all 12 encoder layers, the [CLS] token contains a condensed vector representation (typically of dimension 768) that captures the whole meaning and stylistic characteristics of the entire stanza. This vector is then passed through a classification layer, which is simply a linear transformation that maps the 768-dimensional [CLS] vector to a new vector of 6 values — one for each poet. These values are referred to as logits, representing the normalized scores for each class.

   Finally, in the output layer, these scores are passed through a softmax function, which transforms them into a probability distribution over the six poet classes. The class with the highest probability is selected as the predicted author.

The classification probability is denoted as:

$$P(y \mid input) = softmax(W \cdot CLS + b)$$

And the predicted poet is:

$$predicted_label = argimax\ softmax(yi)$$

Where y_i the unnormalized score (logit) for class i , and $arg\ max$ the index of the class with the highest probability is returned.

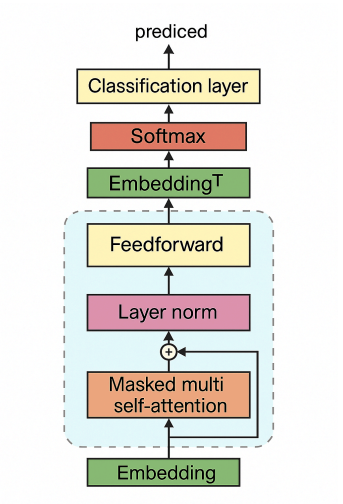

Figure 13 — XLM-RoBERTa arhitecture

Fine-tuning XLM-RoBERTa for Poetic Style Classification

To adapt the multilingual XLM-RoBERTa model for the task of poetic author classification, the prebuilt XLMRobertaForSequenceClassification class provided by the HuggingFace Transformers library was used.

Fine-tuning was managed using the Trainer class, which handles the training loop, loss computation (in this case, cross-entropy), and optimizer configuration. Internally, the `.train()` method performs multiple passes over the training dataset (epochs) and applies backpropagation to adjust the model's parameters so that the [CLS] vector aligns more closely with the target class (author label).

This process enables the model to associate subtle lexical and stylistic features with specific poets. The dataset was divided into training, validation, and test sets, and passed to a custom PyTorch Dataset class that aligned tokenized inputs with their corresponding numeric author labels.

Below is a representative configuration of the training setup:

```
TrainingArguments(
    output_dir="./results_xlm",
    num_train_epochs=5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    logging_dir="./logs",
    logging_steps=10,
    do_eval=True,
    save_steps=500,
    save_total_limit=2
```

Listing 14 — Training Classifier

All hyperparameters were selected based on preliminary testing. A batch size of 16 was found to provide a good balance between performance and resource constraints. Five training epochs represented the empirical sweet spot: fewer epochs resulted in underfitting, while more led to overfitting, particularly due to the moderate dataset size. Periodic checkpointing (every 500 steps) was enabled for traceability and to facilitate the resumption of training if needed.

As a result of fine-tuning, the model learned to assign poetic stanzas to one of six authors by analyzing stylistic signals embedded in word choice, structure, rhythm, and diction. The classification accuracy was measured using the validation set and will be discussed in the following section.

3.3 Perfomance Metrics

The evaluation of the models relies on both quantitative metrics commonly used in machine learning and qualitative methods, such as human-centered poem analysis. In this section, all quantitative techniques will be briefly presented and then explained in detail in the Results chapter, along with the qualitative benchmark.

3.3.1 Metrics for Poetic Style Classification

For the poetic style classification model based on XLM-RoBERTa, which was trained to distinguish between six Romanian poets, it is essential to remind the four main components for performance analysis:

- **TP**—the number of samples correctly predicted
- **FP**—the number of samples incorrectly predicted as the class because they belonged in another class

- **FN**—the number of samples that truly belonged to the class but were not classified as such by the model
- **TN**—the samples that did not belong to the class, and the models predicted this correctly

The following standard multi-class classification metrics were used:

- **Accuracy**: measures the proportion of correct predictions. It is calculated as:

$$Acc = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ predictions}$$

- **Precision**: indicates the proportion of correctly predicted samples among all samples that the model labeled as belonging to a specific class. In other words, it answers the question: "When the model predicts a certain poet, how often is it correct?":

$$Precision = \frac{TP}{TP + FP}$$

- **Recall**: measures the proportion of actual samples from a given class that the model correctly identified. It answers: "Out of all poems written by a given poet, how many did the model recognize correctly?":

$$Recall = \frac{TP}{TP + FN}$$

- **F1 Score**: provides a balanced view of a model's performance by combining precision and recall into a single metric. It answers the question: "How well does the model balance correctness (precision) with completeness (recall)?":

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- **Confusion Matrix**: provides a detailed view of the predictions made by the model. It compares the actual and predicted authors for each stanza, where each row represents the true poet and each column represents the predicted one. It answers the obvious question "Which poet was confused with whom?", highlighting stylistic overlaps between authors, and it helps understanding where the model struggles, so it can be explicitly reassessed with what it needs.

3.3.2 Metrics for Poetry Generation

Since text generation is not a classification task, it cannot be directly evaluated using metrics like accuracy. Instead, the following measures were used to assess the fluency, coherence, and stylistic fidelity of the generated poems:

- **Perplexity**: illustrates how well a model predicts a sequence of tokens. Lower perplexity values indicate that the model has overall better predictions in terms of coherence. However, it is essential to note that perplexity does not evaluate meaning,

creativity, or stylistic nuance — it only reflects how predictable the next token is according to the model's internal probability distribution.

$$Perplexity = e^{Loss}$$

- **Training Loss**: Although not typically a formal evaluation metric, training loss is crucial during the fine-tuning of models. It is a measure of how well the model is learning to predict the next word or token at any training step. A training loss that continues to fall suggests that the model is improving at generating text. In other words, it is slowly learning to recognize the features in the training data. This behavior is taken as an indication of stable learning and good convergence. If the loss isn't decreasing, it's increasing, or it's fluctuating, then it may be a sign of undertraining, overtraining, or unstable training.
- **Stylistic Score**: The degree to which the generated poem matches a target style is estimated by analyzing the output of the fine-tuned XLM-RoBERTa classifier for each poem.
- **Self-BLEU and Self-ROUGE**: BLEU prioritizes precision—how much of the generated text is a perfect match to other text—while ROUGE focuses on recall and structural overlap. These measures do not compare the generated poems against any reference poem, but instead, they compare the generated poems to one another. The idea is that if a model consistently generates poems that are too similar, it is uncreative or has a narrow scope. A low Self-BLEU or Self-ROUGE score indicates that the model has generated a diverse range of styles, word choices, and phrasing, rather than repeating specific word displays.

4 Results

In this chapter, the results for the generator and stylistic classification—also used for validating the generation model independently—are explored through graphics and in-depth explanations, following a qualitative study using a question-based form for human identification of the given generated poem or the real author.

4.1 Results for fine-tuned XLM-RoBERTa

This section presents the performance of the fine-tuning process for the XLM-RoBERTa model, followed by an in-depth analysis of performance metrics, including accuracy, precision, F1-score, recall, and a confusion matrix.

4.1.1 Training Monitoring

During training, Weights & Biases was used to monitor learning metrics in real-time, tracking essential elements such as training loss, validation accuracy, and learning rate to better understand how well the model is trained. This was particularly helpful in knowing how to achieve better results for the model.

Figure 15.1 and Figure 15.2 — Monitoring Training XLM-RoBERTa

During the training, the loss indicates a constant and healthy decrease as it it depicted in the Figure 15, showing that the model learns the data efficiently without overfitting, because there is no significant growth or stagnation. The fact that the loss can be easily observed within the first 200 iterations and then stabilizes indicates that the learning setup (optimizer, batch size, learning rate) was well chosen.

Regarding the learning rate, the model employs learning rate decay, where the rate is reduced proportionally to the increase in the global step, thereby helping the model stabilize during training. This allows the model to make significant updates early on and fine-tune its weights more carefully at the end, resulting in a promising classifier.

4.1.2 Inference and Evaluation Metrics

Inference

Inference refers to the application of the classifier in the project; it is the step that follows fine-tuning and precedes the preparation of performance metrics for the classification part. After a poem is generated, it needs to be classified, and the text is automatically split into eight lines. This step is crucial because it ensures that the classification model receives inputs similar in structure and length to those it was trained on. Each stanza is passed through the classifier and predicts the probability distribution over all author classes. Finally, the author indicated that the entire poem is obtained by computing the average probability across all stanzas. This method increases accuracy and reduces noise from stylistically ambiguous fragments.

Classification Report:

Class	Precision	Recall	F1-Score	Support
Ana Blandiana	0.76	0.83	0.80	47
George Bacovia	0.92	0.81	0.86	68
Lucian Blaga	0.93	0.98	0.96	57
Marin Sorescu	0.71	0.80	0.75	76
Mihai Eminescu	0.88	0.77	0.82	75
Tudor Arghezi	0.76	0.76	0.76	78
accuracy			**0.82**	**401**
macro avg	0.83	0.83	0.82	401
weighted avg	0.82	0.82	0.82	401

Table 16 — Classifier overall performance

The model's performance is ashown in Table 16. The classifier was tested on a batch size of 401 stanzas, with each author represented by a well-distributed number, as shown in the support column. The classifier achieves an overall accuracy of **82%** and maintains balanced performance across all classes. Macro and weighted averages are close, indicating that the model does not exhibit bias due to the frequency of courses.

Lucian Blaga achieves the best results, as indicated by its F1-score of 0.96, primarily due to a recall rate of 0.98. This means it recognizes almost every poem perfectly. The model is rarely confused because the precision is 0.93. **George Bacovia** achieves, like Blaga, a promising precision that makes the model correct almost every time, with a recall of 0.86, ranking him second in terms of well-learned style.

Ana Blandiana has a high recall of 0.83 and a precision of 0.76, which suggests that the model accurately identifies her style but sometimes mistakes the poems of other authors as being her work.

Mihai Eminescu is the inverse case of Blandiana; here, the precision is greater than the recall, which means that some of his poems were misclassified as a different style than Eminescu's, but when predicted, the model performs well with high accuracy. In **Tudor Arghezi**'s case, the precision and recall are equal to 0.76, indicating a stable performance, but modest.

Lastly, the model exhibits the lowest precision in the case of **Marin Sorescu,** suggesting that it frequently misattributes poems to other authors. Despite this, the recall remains relatively high (0.80), indicating that the model successfully identifies the majority of his actual poems. This imbalance implies that while the model is sensitive to Sorescu's stylistic patterns, it lacks sufficient specificity in distinguishing them from those of other authors.

Confusion Matrix

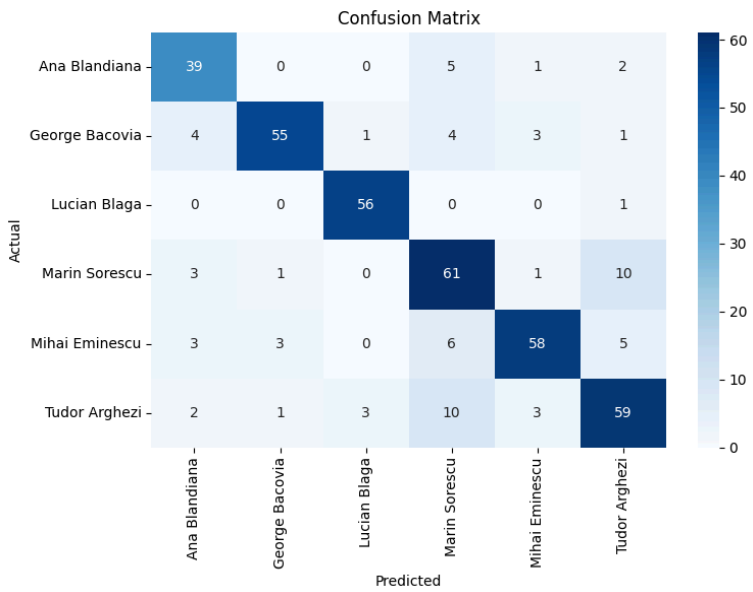

Figure 17 — Confusion Matrix

The confusion matrix provides a detailed illustration of how the model distinguishes between poetic styles. The main diagonal represents the number of correct predictions for each class, while the values outside of the diagonal capture the confusions between authors.

Lucian Blaga, as anticipated from the above analysis, is almost perfectly recognized, having only a single error. The model behaves accordingly, as Blaga has a distinctive style that is widely acknowledged by any avid reader familiar with his work; this is particularly important as it shows that poetic style can be quantitatively modeled and learned, and that the classifier indeed performs exceptionally well.

Marin Sorescu is frequently mistaken for Arghezi due to their stylistic resemblance, which the model notes. Additionally, Mihai Eminescu is occasionally misclassified as Sorescu or Blandiana.

What stands out is that Ana Blandiana's style is not confused with one particular class, but rather with all others, and for her readers, this can be seen as an insightful result. Her style is often perceived as a synthesis of multiple poetic voices.

Indeed, her innovative use of language and thematic fluidity make her work difficult to label or capture through a rigid machine learning framework — a limitation not of the model, but of the inherently versatile nature of her style.

4.2 Results for fine-tuned RoGPT2

This section will explore the training of the model, following the performance of the generative model using metrics described in the previous chapter, such as perplexity, self-ROUGE, and self-BLEU, with examples.

4.2.1 Training Monitoring

The PoetGPT model was also carefully monitored, using the same optimization metrics to gain a better understanding of the model's behavior and inform more informed decision-making when adjusting training arguments.

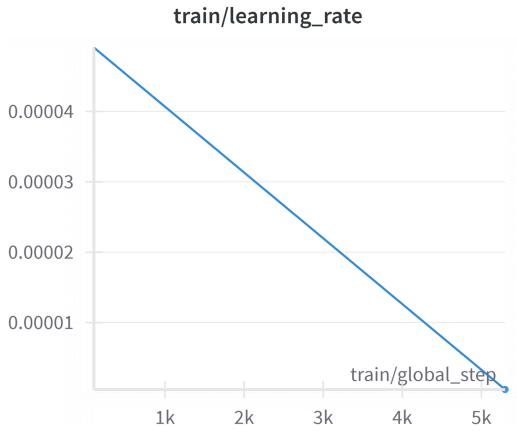

Figure 18.1 and Figure 18.2 — Monitoring Training RoGPT

As shown in Figure 18, the training loss decreases significantly from approximately 4.0 to 2.3 by the end of training. That decrease implies the model has already learned to reduce prediction error in the process of generating the next token, thereby creating sentences that are both more fluent and more sensible.

At the same time, the learning rate was linearly decayed (Figure 18). This means the model made significant learning updates initially, during its starting-to-learn phase, and more minor, conservative updates later on. This slow decrease mechanism can prevent overfitting and stabilize the training process near convergence.

The gradients were also monitored during training; small fluctuations appeared as expected, but their values were within a reasonable range (<3-4.5), meaning AdamW is operating correctly. A value that is too small results in underfitting, and a value that is too large leads to instability, as the optimizer becomes unstable. It is crucial for preventing divergence during backpropagation because the optimizer worked as expected.

Furthermore, the training was smooth, with uninterrupted progression through the training steps and epochs. The five epochs worked best, ensuring the model would not overfit.

4.2.2 Inference and Evaluation Metrics

Inference

During inference, the fine-tuned model was used to generate poems based on a structured prompt:

```
<|autor={chosen author in chat}|>\n<|titlu= chosen title in chat|>\n<|poezie=>
```

This format was defined during the training process and was consistently maintained throughout the generation process, making it a **prompt design** technique. This approach establishes a standardized format in which the model can receive explicit instructions, providing structure that helps activate the correct behavior of the model.

```
output = generator(
    prompt,
    do_sample=True,
    temperature=0.8,
    top_k=50,
    top_p=0.95,
    repetition_penalty=1.1,
    max_new_tokens=512,
    eos_token_id=tokenizer.convert_tokens_to_ids("<|endofpoem|>")
)
```

Listing 19 — Inference Generative Model

In the last layer of the model, the words are chosen based on a greedy deterministic choice or randomly, such as “chosen,” because the purpose is to generate creative text, not the “best” word choice, by setting True to `do_sample`.

The temperature safely adjusts the entropy level of distribution, allowing it to be creative, yet correct because of the values < 1. The model achieves better results when setting top_k=50, meaning it considers only the top 50 most probable words at each generation step. These are selected from a group that holds a high cumulative probability of 95%, as controlled by top_p=0.95.

Repetition is controlled by a repetition_penalty of 1.1, which discourages the model from reusing exact words or phrases. Finally, the value of max_new_tokens=512 allows for sampling longer poems, thereby reducing the probability of obtaining the special <|endofpoem|> token, which ensures the output is generated correctly and the poem has a proper ending.

Perplexity

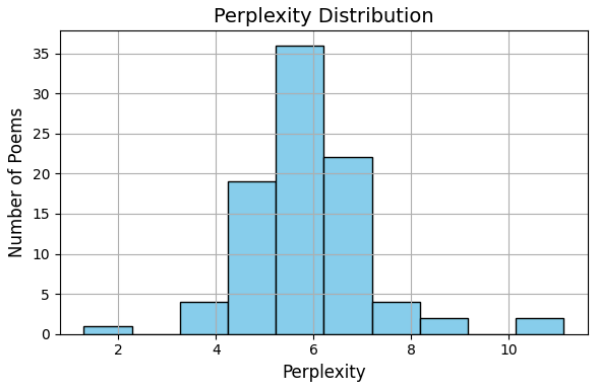

Figure 20 — Perplexity Generative Model

As shown in Figure 20, the values fall between 5 and 7, with a peak at 6, indicating that the model produces fluent and coherent text.

Values between 1 and 3 are assigned to poems that are extremely fluent but almost copied from the original, which indicates overfitting; however, this is not the case here.

Values like 8-10 indicate an unpredictable poem, even innovative, but with coherence and fluency that are not entirely natural; this is not a problem for a creative text.

The sweet spot, 4-7, which the majority of the poems fall into, indicates natural language that is fluent yet still creative and original. Very few poems are incoherent, where the model is unsure or has errors, as shown below (>10).

Self-Bleu and Self-ROUGE

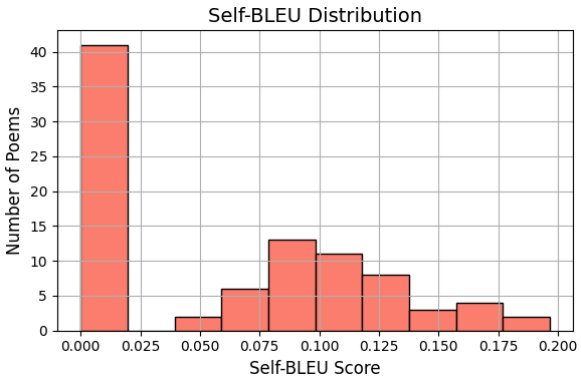

Figure 21

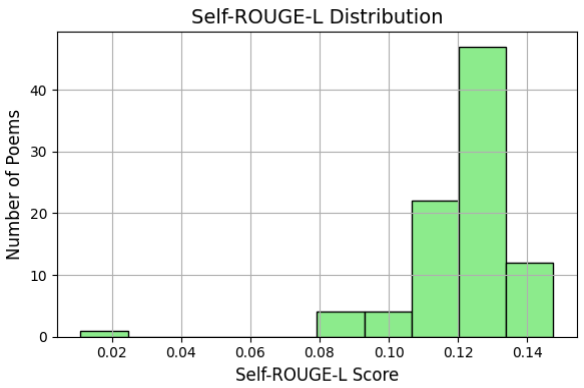

Figure 22

Figure 21 shows that the Self-BLEU distribution yields promising results, with the poems scoring between 0.00 and 0.10, indicating a high degree of diversity that can be interpreted as a sign of creativity. The poems do repeat one another, even the highest scores (0.12–0.18) remain within an acceptable range, but may indicate some stylistic overlap.

Figure 22 illustrates the Self-ROUGE distribution, specifically the “L” type, which measures structural similarity based on the longest common subsequence between generated texts. Most poems fall within a similarity range of 0.11 to 0.14, a relatively low score suggesting that the model produces poems that differ not only in word diversity (as shown by Self-BLEU) but also in sentence composition and rhythm.

4.2.3 Style Accuracy

Poem style classification is a challenging task to be mathematically modeled; the classifier offers an approximation as it is trained on genuine poems, so the style accuracy becomes a proxy metric — it does not measure directly the “literary style,” which is a subjective problem, but it offers an objective, automatic indicator of style.

Usage of the style classifier

To ensure style consistency, the generation process included a validation loop. Specifically, it used a function that:

- generates a poem based on the desired author and prompt,
- classifies the poem's stanzas using the XLM-RoBERTa model,
- calculates the average style score for the intended author,
- regenerates the poem up to 10 times if the target author's score is below a set threshold (70%),
- selects the best candidate based on the highest average style score for the requested author.

Data for validation

To evaluate the stylistic accuracy of poems generated by the fine-tuned PoetGPT model, a controlled experiment was conducted utilizing a stylistic validation mechanism. For each of the 6 Romanian authors, a list of 21 different prompts — poetic titles designed to be stylistically neutral- was provided. The model generated a poem for each author-title pair, resulting in a total of 126 poems.

Measurement

Style accuracy was calculated by comparing the intended author (from the prompt) with the author predicted by a stylistic classifier trained on genuine poems. For each of the 126 generated poems, the classifier returned a probability distribution over the six authors, and the author with the highest score was selected. A prediction was considered correct if it matched the prompt author. The overall style accuracy obtained was **82.17%**, indicating that the model successfully captured and reproduced authorial style in most cases.

Stylistic Accuracy per Author

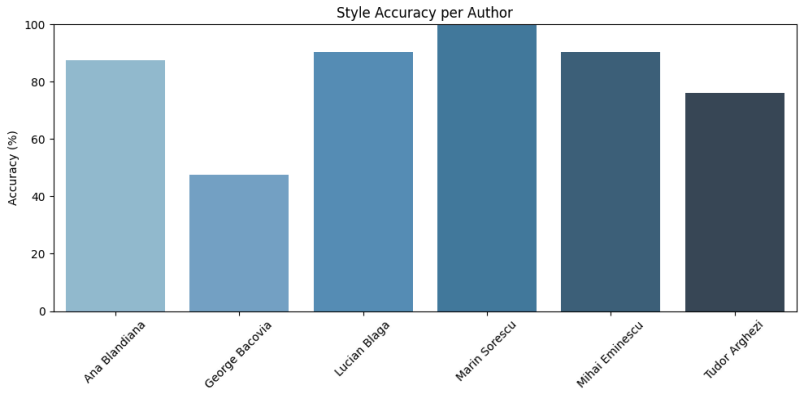

Figure 23

The chart in Figure 23 shows the percentage of generated poems that were classified as belonging to the intended author, as specified in the generation prompt.

The highest style accuracy was achieved for Marin Sorescu, Lucian Blaga, and Mihai Eminescu, each exceeding 90%. What is interesting is that this result differs from the classifier's performance on genuine (test) poems, where Sorescu ranked the lowest, along with Blandiana.

Misclassification Patterns between Authors

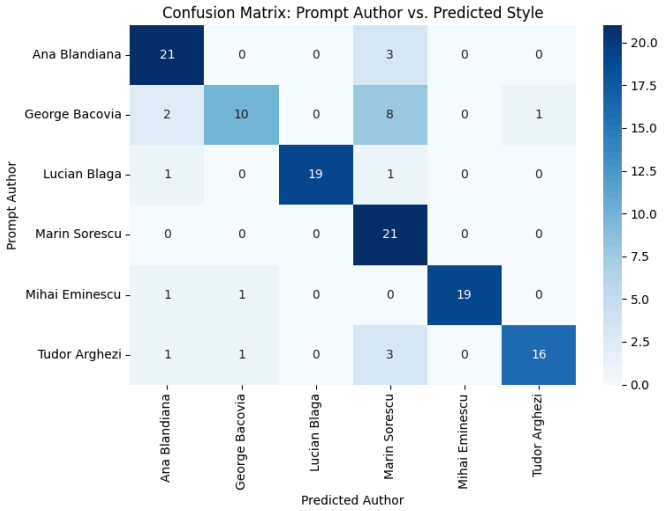

Figure 24

The confusion matrix in Figure 24 highlights the problems and limits that the generation models have. Even though, as shown on the main diagonal, there are high values for Sorescu, Eminescu, and Blaga, which suggests that the model generates recognizable poems with authors from the prompt, Bacovia is not one of them.

This indicates that the model struggles with subtle, minimalist traits, which, for example, Bacovia possesses. Overall, the poems generated are stylistically accurate.

Style Score Distribution

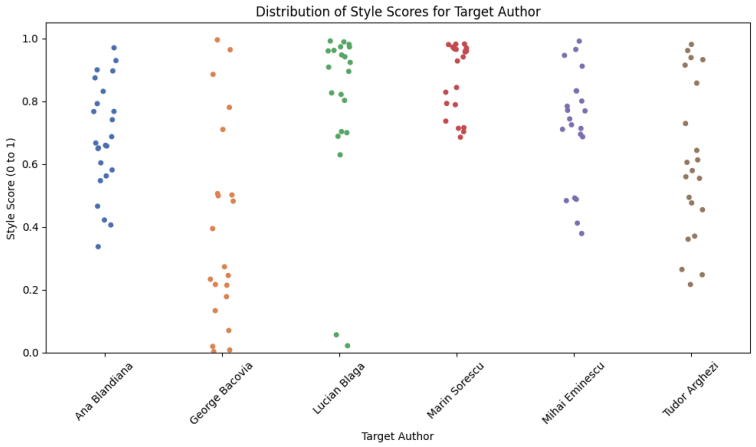

Figure 25

The distribution in Figure 25 captures an additional perspective on the coherent and stylistic force of the generated poems. The distribution shows for each author what score each generated poem received when stylistically validated, and this matters to observe not only if a poem has a stylistically accurate style, but also the generator's ability to create consistently stylistically accurate poems for each author. This is a crucial indicator of the generator's global quality.

The strip plot outlines the generator's deficiency in imitating Bacovia, as many of the scores are under 0.5, but the majority is above 0.85, indicating a consistent and convincing style. Blandiana and Arghezi have the most challenging style to reproduce consistently, as the tipar is difficult to replicate, but it most often yields beautiful results.

4.3 Qualitative Evaluation with human participants

To support the automated style validation and the overall quality of the generated poems, a two-part human evaluation was conducted with 40 participants. In the first part, participants try to identify the generated poem between two poems: one is the real work of an author, and the other is a stylistically accurate poem by the same author. In the second part, they classify the poems generated by the model and try to answer the question "Which author's voice does these poems resemble?"

4.3.1 AI-Generated vs. Real Poetry

In the first phase, two poems per author were presented (one written by the author and one generated by the PoemGPT in the author's voice).

They then indicated which poem had been written by the actual poet, justifying their answer, and rated how much they liked each poem (on a 1-to-5 scale). This was designed to assess the perceived realism of the generated texts (a "Turing test" for poetry) as well as their aesthetic quality.

George Bacovia

Interestingly, some of the human responses contrasted with classifier-based stylistic accuracy. For instance, although Bacovia's generated poem had one of the lowest style accuracy scores in automatic validation (around 61%), 75% of participants mistakenly believed the generated poem was real.

Furthermore, it received a slightly higher appreciation score than the actual one (3.42 vs. 3.35). This suggests that Bacovia's abstract, fragmented, and melancholic tone can be easily mimicked at the surface level and may seem convincing to non-expert readers. However, expert participants noted issues such as a lack of unpolished musicality and emotion.

Tudor Arghezi and Ana Blandiana

Conversely, for Arghezi, the AI poem scored significantly lower in appreciation (3.35 vs. 4.1) and was correctly identified as fake in 65% of cases.

Participants noted the lack of archaic diction, the missing tension in voice, and metaphor overload — all traits that deviate from Arghezi's sharper tone. This aligns with the classifier results, which showed Arghezi to be moderately complex to model (F1-score of 0.76) and frequently confused with Blaga or Blandiana.

For Ana Blandiana, the generated and real received averages, respectively, were 3.75 and 3.92, with 60% of the participants correctly identifying the generated poem.

Respondents noted that the style drew musical inspiration from the lyrics and that the musical ideas at the heart were presented in general terms, with some metaphors not conveying precise meanings. This writer's style was frequently praised for its thematic fluidity and poetic vagueness, making the generator not better than the original, but close and coherent.

Lucian Blaga and Mihai Eminescu

Lucian Blaga's generated poem also closely matched the real one in appreciation (3.38 vs. 3.95), with 57% of participants correctly identifying the fake. Interestingly, several comments noted that the generated version captured Blaga's key imagery and depth but failed to maintain his metaphysical consistency, a trait identified only by participants with prior knowledge about the particularities of his style.

This is consistent with the classifier's stellar performance on genuine Blaga poems (F1-score: 0.96) and high style accuracy (>90%) for generated ones. The study results prove that PoetGPT generates as expected by the automatic evaluation metrics, being consistent, fluent, and metaphorical.

Mihai Eminescu's generated poem received a nearly identical rating to the real one (3.9 vs. 4.0) and was correctly identified as AI-generated by 57% of participants. Those with more knowledge of literary theory, particularly regarding Romanticism and Modernism, pointed out the lack of rhyme and archaic language as signs of the poem's artificial nature. However, they also agreed that it still has the Eminescu lyrical tone, as well as his well-known themes, motives, and repetition, which are familiar to everyone.

Marin Sorescu

Marin Sorescu Finally, Marin Sorescu, who is one of the most retro-romancesque authors (>92% style accuracy), was very popular among the readers. The resulting poem scored 3.48 (w.r.t 3.85 real) and was well classified as fake in 57%. Its ambiguity and metaphorical language were commented on as typical aspects of Sorescu's recognizable style, credit which can be added to the accuracy of the classifier and the efficiency of the generator in his case.

4.3.2 Style Attribution

In the second part, participants were shown a single generated poem and asked to choose the most likely author from the six known poets. This setup evaluated intuitive stylistic recognition without comparison to a real poem.

The results largely mirrored classifier-based accuracy trends:

- Sorescu was correctly selected in 35% of cases, with notable confusion toward Arghezi (22%) and Blaga (17%) — indicating stylistic overlap.
- Bacovia, though weak in classifier validation (61%), was correctly guessed by 70% of human participants, suggesting that its abstract style has strong perceptual markers.
- Eminescu was correctly identified in over 65% of responses, with minimal confusion (the next closest was Blandiana at 20%). This confirms the recognizability of his Romantic voice.
- Blaga was chosen correctly in 45% of cases, while the remaining votes were evenly distributed — a sign of nuanced, less biased misclassification.
- Arghezi was harder to identify: only 25% selected him, while 22.5% picked Blaga and 20% Blandiana. This confusion reflects overlapping registers and shared symbolic intensity.
- Blandiana was the most misclassified: only 30% of participants chose her, while 35% guessed Arghezi and 20% Sorescu. This matches classifier behavior and confirms that both humans and machines struggle with her stylistic variety.

4.3.3 Conclusion of the study

These results confirm that the poetic generator developed in this work—PoetGPT—is capable of producing stylistically convincing texts, particularly when paired with a robust, independently trained style classifier.

The stylistic accuracy metric introduced here, based on automatic author attribution, proves not only scalable and consistent, but also aligned in key areas with human perception. Sorescu, Blaga, and Eminescu emerge as strong voices in both automatic classification and reader intuition, while Bacovia reveals a unique discrepancy: although the classifier scores his generated poems lower, participants overwhelmingly perceived them as authentic. Conversely, Arghezi and Blandiana illustrate the limits of both machine and human detection, confirming the complexity and ambiguity of their styles.

5 Discussion

This chapter presents a critical overview of the model, encompassing both the generative model and the style classifier. The first part is dedicated to a performance comparison of the results obtained in personal experiments while choosing the best model, including a more detailed explanation with an ablation study for the classifier. The second part highlights the main limitations of the current method, specifically in terms of data, architecture, or the quality of the generated poems.

5.1 Performance Comparison

5.1.1 Generative models

To generate poetry, several autoregressive language models were explored, comparing both English-based and Romanian-based architectures. The goal was to identify the configuration that delivers the most stylistically faithful outputs for canonical Romanian authors.

Fine-tuning stages

Initially, the first experiment used DistilGPT, a smaller and faster model pre-trained in English. The model was fine-tuned on a subset of the Romanian poetic corpus, but the results were incoherent and grammatically incorrect, indicating poor language adaptation.

The best choice was RoGPT2, a Romanian autoregressive language model trained on two versions:

- The first version was fine-tuned without the use of special tokens. The text was grammatically correct, but lacked a poetic structure. The generated text contained even the author's name and title, and showed an apparent lack of understanding of the lyrical structure.
- The second version included tokens such as <|author=...|>, <|title=...|>, <|poem=...|>, and <|endofpoem|>. This led to significant improvements: the poems reflected the author's specific tone and vocabulary, and the model learned to generate natural endings.

Prompt design and its impact on quality

Prompt design played a key role in shaping the structure and fluency of the generated texts. Several variations were experimented with to find the best one:

- Including the title as the first line of the poem (e.g., <|poem=The Autumn Fog\n...) resulted in lower quality. The model treated the title as part of the content, breaking the stylistic coherence.
- Adding semantic tags like <|tone=melancholic|> and <|theme=death|> led to inconsistent results. Since the model had not seen these during training, it sometimes followed the tone, but often ignored the cue or extended it incorrectly in the generated text.

5.1.2 Ablation Study: Poetry Style Classifier

An ablation study is a method used in machine learning to determine the individual impact of different components of a system. It involves a systematic analysis of the model whenever an element is changed or adapted to observe what works for the model and what does not.

In this project, a comprehensive ablation study was conducted for the poetic style classifier, aiming to understand how architecture, input format, stylistic features, and model complexity affect classification performance across six Romanian authors, as well as determining the best model to use for a valid style accuracy measurement of the generated poem.

Model Setup	Accuracy %	F1-Score
XLM-R default	68	0.65
XLM-R + [CLS + Style Vector]	27	0.19
XLM-R + Style Vector + Tuned Param	53	0.50
XLM + Deep MLM	40	0.37
Simplified MLP + Dropout 0.1	50	0.50
BERT Multilingual(Text Only)	48.38	0.52
BERT + Style Vector Input	23	0.2
XLM-R SeqClassifier + Strophes	70	0.66
XLM-R SeqClassifier + Style Vector in DF	82	0.78

Table 26 - Ablation study Classifier

XLM-R default

Using the standard xlm-roberta-base with its built-in classification head (a single linear layer) yielded solid results. This setup treats the [CLS] token — a global representation of the input

— as the basis for classification across six authors. Despite not using any handcrafted features or data engineering, the model captured many stylistic patterns.

However, since input texts were complete poems, long examples (e.g., Luceafărul) were truncated at 512 tokens, which removed important contextual and stylistic information. As a result, classification errors often occurred between authors with overlapping themes or tone, notably Bacovia and Blaga.

XLM-R + [CLS + Style Vector]

In this version, the handcrafted style vector is used for the [CLS] embedding. This meant feeding the model a hybrid input: part from the transformer (CLS), part from external numeric data. The Wang's project inspired this idea, utilizing Chinese poetry (Wang, 2022),, where style vectors significantly improve the accuracy.

Despite the intuitive design, performance collapsed to 27% accuracy. The model struggled to reconcile textual and numerical modalities, possibly because the [CLS] embedding is high-dimensional (768d), while the style vector was just 6d, creating an imbalance. Moreover, no extra layers were added to process the fusion properly. This highlights the importance of accurately adapting innovative ideas to their specific context and utilizing effective architectural design.

XLM-R + Style Vector + FC layer + Tuned Parameters

To address the mismatch between the [CLS] embedding (768-dimensional, semantic) and the handcrafted style vector (6-dimensional, numeric), several architectural and training adjustments were enabled.

The most significant change was the addition of a fully connected (FC) layer with 256 neurons, following the concatenation of the [CLS] vector and the style vector. A fully connected layer connects every neuron from the previous layer to every neuron in the current layer, which is salient for the model to learn weighted combinations of input features, allowing it to reinterpret the given input in a more abstract and unified representation space.

```
1. Before: [CLS] (768d) → Linear(768 → 6)
2. After: [CLS] + style (774d) → Linear(774 → 256) → ReLU → Linear(256 → 6)
```

Listing 27 — Before and After Layers

This was necessary because the concatenated input (774 dimensions in total) combined two very different types of information:

- A rich, distributed semantic vector from the transformer encoder ([CLS]), and

- A compact, structured numeric vector of style features (verse count, adjective ratio, punctuation, rhyme, etc).

Without a transformation layer, the model would treat all 774 values equally, even though the last 6 represent a completely different modality. The FC layer acts as a fusion mechanism, enabling the model to learn how to map and combine textual and stylistic signals meaningfully before sending them to the classification head.

To maintain this architectural change, some regularisation was required:

- Increased the number of training epochs from 4 to 8 to let the model see more examples to learn those stylistic subtleties.
- Lowered learning rate (1e-5) such that fine-tuning makes more minor, more consistent updates to the parameters.
- To avoid gradient explosion and stabilize training dynamics, it was implied to use gradient clipping (max_norm = 1.0).
- Explicit dropout was employed for the style vector part of the input, which pushed the network to learn more general patterns rather than memorize the input patterns.

Together, these adjustments increased the model's accuracy from 27% (when simply concatenating the style vector) to 53%. This result demonstrated that style vectors can be beneficial, but only when they are carefully integrated through appropriate architecture and regularization.

XLM-R + Deep MLP

In this version, the simple classification layer was replaced entirely with a multilayer perceptron (MLP) — a deeper neural network composed of two hidden layers, not just extended with a fully connected layer only (FC).

```
1. Before:      [CLS] (768d) → Linear(768 → 6)
2. After:       [CLS] + style (774d) → Linear(774 → 256) → ReLU →
3.               → Linear(256 → 128) → ReLU → Linear(128 → 6)
```

Listing 28 — Before and After Layers

The idea was to allow the model to learn more complex interactions between the textual embedding and the handcrafted style features.

An MLP (Multilayer Perceptron) applies multiple fully connected layers, each followed by an activation function (typically ReLU), to progressively transform the input into a decision space. While adding a single fully connected (FC) layer helped bridge the gap between the [CLS] embedding and the style vector, it still relied on a relatively shallow transformation. To

further enhance the model's capacity to capture nonlinear, multi-level interactions between semantic and stylistic features, the MLP seemed worth trying..

The rationale was that deeper networks are theoretically better suited to learning complex feature hierarchies, especially when dealing with two modalities of input — text and structured numeric vectors. By adding a hidden layer (256 → 128 → 6), the model was expected to generalize better, extract richer representations, and improve classification accuracy across all authors.

However, this added depth came with a cost:

- The dataset (~900 poems) was too small, and the model overfitted quickly.
- The dropout rate of 0.3, which was too aggressive, resulted in the loss of important information during training.

As a result, accuracy dropped to 40%, indicating that excessive architectural complexity, without sufficient data, actually hinders performance rather than enhancing it.

Simplified MLP + Dropout 0.1

To fix overfitting, the following solutions were tried:

- Reduced the MLP to a single hidden layer (128 neurons).
- Lowered dropout to 0.1 to preserve more information during training.
- Trained for 12 epochs to allow slower convergence.

This balanced configuration brought performance back up to 50%, with some authors (like Blaga and Eminescu) classified reliably. However, modern poets like Sorescu and Blandiana remain difficult to identify, suggesting that their styles require deeper or more abstract features.

BERT Multilingual (Text Only)

Using bert-base-multilingual-cased, a similar setup as the XLM-R default solution was used**,** but with a different encoder. Without any style vector or architectural changes, this model achieved an accuracy of 48.38%.

Interestingly, Blaga, Bacovia, and Arghezi were recognized with high F1 (>0.6), confirming that even pure text contains valuable stylistic signals. Yet the model struggled with subtle or modern styles, especially when poems lacked clear structural cues, such as rhyme or punctuation density.

BERT + Style Vector Input

Here, the style vector was added as a separate column in the input dataset, with the hope that the model would learn to combine textual and numerical features during training.

However, the architecture didn't explicitly know how to connect these — no layer was added to fuse text and numbers.

Result: 23% accuracy, with F1 = 0 for most authors. This demonstrates that simply injecting numbers into the dataset is insufficient — the model architecture must be aware of and adapted to mixed modalities.

XLM-R SeqClassifier + Strophes

One of the most impactful ideas was switching from complete poems to strophe-level input (4–8 lines). This ensured shorter, semantically coherent inputs that fit within the 512-token limit, removing the need for truncation.

Using Hugging Face's XLMRobertaForSequenceClassification with default settings and no additional layers, the model achieved 70% accuracy, with a strong balance of F1 scores. This confirmed that the data structure matters as much as the model architecture.

XLM-R SeqClassifier + Style Vector in Dataset (Final Model)

In the final and best-performing setup after applying the gained knowledge in the ablation study:

- Stanzas were used as input.
- Style vectors were directly included in the dataset (as normalized, preprocessed numeric features).
- Vectors were not concatenated into the model — instead, they allowed the training pipeline to ingest them as part of each example's context.

Accuracy improved as described in the previous chapters, reaching 82%, and the macro F1 score rose to ~0.78. Importantly, performance was balanced across all six authors, with Blandiana and Sorescu no longer being outliers. This setup confirmed that minimal architectural complexity, combined with intelligent data integration, yields the best results in low-resource tasks.

5.2 Limitations

Although the results indicate significant progress in generating and classifying poems, it is essential to analyze critically the limitations that the methodology presents. This section presents the main constraints identified during the experiments, categorized into three areas: limitations of the available data, model architecture, and the last one concerning stylistic quality and conceptual understanding of style. The feedback in the study, which offered a human evaluation of the generated poems, helped reveal a more nuanced perspective on the aspects that remain problematic in creative text generation.

5.2.1 Dataset limitations

The data volume consists of almost 150 poems per author, totaling 918, a modest dataset given the complexity of the tasks. Although the distribution per author was balanced, the reduced dimension of the dataset limited the model's capacity to learn fine stylistic traits. Moreover, the lack of semantic diversity by augmentation, paraphrasing, or introduction of thematic variations limited the variety of poems from which the model could have learned better semantic relations and stylistic features.

Moreover, the dataset only contains information about the author, title, text, and, for the classifier, the style vector. Information about the historical period, literary current, and predominant themes would have offered a more profound learning of the poetic language.

5.2.2 Architectural limitations

The final adopted architecture in the project — RoGPT2 for poem generation and XLM-RoBERTa for Sequence Classification with the stylistic vector in the dataset, not a neural network — yielded the best score: 82% style accuracy and fluent text generation. Although this simplified method has necessary architectural limitations, it also highlights the need for and ideas about future work in the project.

In this form, the style classifier does not explicitly fuse the textual information with the numeric one — the style vector. This vector is added as a new column in the dataframe, but not concatenated with the embedding [CLS] or processed in the network.

This way the model does access the information, but indirectly associates them with their etiquettes during fine-tuning. Although this yielded the best results, it does not guarantee that the style is learned or understood; rather, it only indicates that the model identified useful correlations between style and author.

In contrast, a more integrated architecture — for example, the ones analyzed in the Discussion section — such as explicitly concatenating the vector to the [CLS], followed by a fully connected layer or an MLP, would, in theory, have shown better results.

In these experiments, where the limitation was attempted to be overcome, others were found that hindered the model's performance, resulting in accuracy levels of 27% or 50% in the best case.

Moreover, the two components of the system work completely separately. The interaction between them is external and more akin to a filter rule, serving as a personalized evaluation metric. There is no feedback mechanism or internal adjustment during generation with the help of the style classifier, which limits the potential for automatic adaptability and real-time stylistic refinement.

5.2.3 Stylistic and conceptual limitations

While the generated texts appear to follow a consistent poetic structure, human evaluation revealed significant deficiencies in expressiveness and style coherence. The poems do not always convey their emotional tone clearly, and the rhythm or inflection shifts abruptly between words on the page.

The stylistic devices can be somewhat mechanical, lacking much poetic sense, and commonly, the vocabulary of the author's known period is missing, with no beneficial effect on style fidelity. Plus, rhymes don't always come naturally, and repeating the same tactics is not as discerning as poetic language that might not repeat itself in precisely the same way.

These observations point to a deeper conceptual challenge: The style of literature is not a function of form or statistics alone. It's about emotion, intent, aesthetic message, and cultural context—things that a Transformer-based model doesn't understand intrinsically. Even if one is successful at distinguishing among authors based on these lexical and structural cues, such a conclusion would not capture the actual "emotion" or "aesthetic" of poetry. Therefore, the problem of expressing a real artistic idea instead of imitating form remained unsolved.

6 Conclusions and Future Work

6.1 Future Work

Future improvements will likely involve a deeper integration between linguistic generation and literary understanding. One possible direction is training a model not only on poems, but also on literary criticism and theoretical writings about style, themes, and periods. Such a model could act as a "literary mentor," guiding poetic generation with knowledge drawn from the author's broader oeuvre and the interpretive frameworks of literary theory.

In terms of architecture, future systems could combine the generator and the classifier into a unified, feedback-driven model that adapts generation based on live stylistic evaluation. Style vectors could also be embedded directly in the model, not just at the data level, to enable richer semantic fusion. Additionally, building a larger and more diverse dataset — annotated by literary period, emotion, tone, or figures of speech — could help capture the deeper stylistic identity of a poem, beyond surface patterns. Poetic generation could evolve from author mimicry to stylistic understanding, moving from formal replication to creative emulation, guided by both neural training and literary insight, by integrating sentiment, enabling the generation of emotionally coherent and thematically consistent poems. This approach would bring neural poetry generation one step closer to genuine artistic authorship.

Extending human studies remains essential. Future research should involve larger and more diverse participant groups, including experts in literature and poetry, as well as more complex studies in which readers assess and analyse poems.

In summary, the future development of PoetGPT should focus on a deeper modeling of poetic structure, a more explicit representation of authorial style, and tighter feedback loops between generation and evaluation. It would enable better stylistic fidelity but would also make essay submission systems more expressive—the tool could help write poems, analyze poems, or even simulate an authorship.

6.2 Conclusions

PoetGPT is a hybrid system that addresses a unique challenge: generating poetry in the styles of six Romanian authors and validating its stylistic authenticity through machine learning. Beyond its technical contribution, the project offers a deeper reflection on what it means to mathematically approximate something as elusive as literary style — and where the boundary lies between imitation and authentic artistic expression.

Through fine-tuning, prompt conditioning, and classifier feedback, this system examined the extent to which neural networks can model an author's voice. At its core, this thesis is not only a software implementation but also a complex inquiry into the limits and possibilities of artificial creativity.

The results are promising, even if they demonstrate the current impossibility of feeling, intention, musicality, or symbolic meaning in AI text, aspects that are crucial in human poetry. However, it is precisely within these constraints that new opportunities emerge: PoetGPT paves the way for more comprehensive integrations between language models and literary knowledge, thereby encouraging future systems to learn from theory, criticism, and human interpretation, in addition to text.

In this light, PoetGPT becomes more than a generative tool — it becomes a case study on how creative language can be simulated, evaluated, and eventually, reimagined through the lens of computational artistry.

-

References

(Lau et al., 2018) Lau, J. H., Cohn, T., Baldwin, T., Brooke, J., & Hammond, A. (2018). Deep-speare: A joint neural model of poetic language, meter, and rhyme. arXiv preprint arXiv:1807.03491.

(Wang, 2022) Wang, C. (2022). Analysis of poetry style based on text classification algorithm. Scientific Programming, 2022(1), 2763380.

(Uthus et al., 2021) Uthus, D., Voitovich, M., & Mical, R. J. (2021). Augmenting poetry composition with verse by verse. arXiv preprint arXiv:2103.17205.

(Calin, 2020) Calin, O. (2020). Statistics and machine learning experiments in english and romanian poetry. Sci, 2(4), 92.

(Zhao, 2023) Zhao, W. (2023). On the Principles of Evaluation for Natural Language Generation (Doctoral dissertation, Dissertation, Darmstadt, Technische Universität Darmstadt, 2022).

(Manning, 1999) Manning, C. D. Schütze. H.(1999). Foundations of statistical natural language processing.

(Vaswani et al., 2017) Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.

(Devlin et al., 2018) Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NACL 2018), Minneapolis, Minnesota, USA.

(Niculescu et al., 2021) Niculescu, M. A., Ruseti, S., & Dascalu, M. (2021, November). Rogpt2: Romanian gpt2 for text generation. In 2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI) (pp. 1154-1161). IEEE.

(Wiciaputra et al., 2021) Wiciaputra, Y. K., Young, J. C., & Rusli, A. (2021). Bilingual Text Classification in English and Indonesian via Transfer Learning using XLM-RoBERTa. International Journal of Advances in Soft Computing & Its Applications, 13(3).

(Prytula, 2024) Prytula, M. (2024). Fine-tuning BERT, DistilBERT, XLM-RoBERTa and Ukr-RoBERTa models for sentiment analysis of Ukrainian language reviews. Machine learning, 3(4).

(Zeng et al., 2024) Zeng, Z., Han, Z., Ye, J., Tan, Y., Cao, H., Li, Z., & Huang, R. (2024). A conspiracy theory text detection method based on RoBERTa and XLM-RoBERTa models. Working Notes of CLEF.

(Gao et al., 2023) Gao, M., Ahipasaoglu, S., & Schuster, K. (2023). Automated poetry scoring using BERT with multi-scale poetry representation. International Journal of Intelligent Systems Technologies and Applications, 21(3), 250-261.

(Elson et al., 2010) Elson, D., Dames, N., & McKeown, K. (2010, July). Extracting social networks from literary fiction. In Proceedings of the 48th annual meeting of the association for computational linguistics (pp. 138-147).

(Saddhono et al., 2024) Saddhono, K., Saputra, N., Saragih, B., Sumbayak, D. M., & Sipayung, R. W. (2024, May). A New way to Create Virtual Authors/Writers using AI Based Technology with Optimized Voice and Style. In 2024 4th International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE) (pp. 1509-1514). IEEE.

(Yazid et al., 2024) Yazid, R., Mustofa, M., & Fitriyah, U. (2024). CAN AUTOMATIC POETRY GENERATION INFUSE VALUES? UNVEILING INSIGHTS THROUGH CONTENT ANALYSIS OF GENERATED POETRY. LiNGUA, 19(1).

(López, 2024) Macías López, Á. (2024). Computational analysis of the prosodic characteristics of the poets' voice.

(Shahriar et al., 2022) Shahriar, S., & Al Roken, N. (2022). How can generative adversarial networks impact computer generated art? Insights from poetry to melody conversion. International Journal of Information Management Data Insights, 2(1), 100066.

(Colton et al., 2012) Colton, S., Goodwin, J., & Veale, T. (2012, May). Full-FACE Poetry Generation. In ICCC (pp. 95-102).

(Agarwal et al., 2020) Agarwal, R., & Kann, K. (2020). Acrostic poem generation. arXiv preprint arXiv:2010.02239.

(Qu et al., 2020) Qu, Y., Liu, P., Song, W., Liu, L., & Cheng, M. (2020, July). A text generation and prediction system: pre-training on new corpora using BERT and GPT-2. In 2020 IEEE 10th international conference on electronics information and emergency communication (ICEIEC) (pp. 323-326). IEEE.

(Dakhore et al., 2024) Dakhore, M., Eti, M., Diwakar, M., Sivanantham, A., Verma, L., & Shyam, M. (2024, December). Blending the Powers of BERT and Neural Style Transfer for Artistic Text Generation in Poetry. In 2024 IEEE 2nd International Conference on Innovations in High Speed Communication and Signal Processing (IHCSP) (pp. 1-6). IEEE.

(Shao et al., 2021) Shao, Y., Shao, T., Wang, M., Wang, P., & Gao, J. (2021, October). A sentiment and style controllable approach for chinese poetry generation. In Proceedings of the 30th ACM international conference on information & knowledge management (pp. 4784-4788).